
TP 7 (programmation) - Chaînes de caractères.

Comme d'habitude, vous devez rendre votre TP dans le délai indiqué. Lorsqu'un fichier contenant les en-têtes des fonctions et des doctests est fourni, vous devez le compléter. Sinon, vous devez écrire un programme par exercice, avec comme nom **exoX.py** (X est le numéro de l'exercice) et mettre en commentaires les tests que vous avez faits pour vérifier que votre programme fonctionne.

Exercice 1

Complétez le fichier `exo1.py` fourni.

1. On souhaite déclarer et afficher la chaîne de caractères suivante (une seule chaîne) :

```
Bonjour
le "Monde"
et l'Univers
```

- (a) Déclarez une première version `s1` utilisant le caractère de retour à la ligne.
 - (b) Déclarez une deuxième version `s2` sans utiliser le caractère de retour à la ligne.
 - (c) Vérifiez que les deux chaînes sont identiques.
2. Écrivez la fonction `est_palindrome` qui teste si un mot (sans espaces) est un palindrome. Vous n'avez pas le droit d'utiliser `reverse`.
 3. **Pour les 3 questions suivantes, vous n'avez pas le droit d'utiliser les fonctions standard sur les chaînes (sauf `len`).**
 - (a) Écrivez une fonction `decoupe_simple(chaine)` qui renvoie la liste des mots contenus dans `chaine`. Ici, on considère que `chaine` contient au moins un caractère, qu'elle ne commence pas ou ne finit pas par un espace et que les mots ne peuvent être séparés que par 1 seul espace.
 - (b) Écrivez une fonction `decoupe_espaces(chaine)` qui renvoie la liste des mots contenus dans `chaine` sachant qu'ils peuvent être séparés par 1 ou plusieurs espaces et qu'il peut y avoir des espaces au début ou à la fin.
 - (c) Écrivez une fonction `decoupe(chaine)` qui renvoie la liste des mots contenus dans `chaine` (ils peuvent être séparés par des espaces, des tabulations, des retours à la ligne, ...). Pensez à utiliser les groupes de caractères prédéfinis : <https://docs.python.org/3.4/library/string.html#string-constants>
Remarque : cette fonction permet d'écrire directement `decoupe_simple` et `decoupe_espaces`.
 4. Écrivez une fonction `decoupe2(chaine)` qui fait la même chose que la précédente, mais cette fois, vous pouvez utiliser les fonctions standard : <https://docs.python.org/3.4/library/stdtypes.html?highlight=str#string-methods>.
Remarque : à partir de maintenant, et sauf indication du contraire, pensez à regarder dans la documentation si ce dont vous avez besoin n'existe pas déjà...
 5. Écrivez la fonction `est_palindrome2` qui teste si une phrase est un palindrome en ne tenant pas compte des espaces ni de la ponctuation.

Exercice 2

Complétez le fichier `exo2.py` fourni. N'oubliez pas que vous pouvez faire vos propres tests. Dans la mesure du possible, vous devez éviter de dupliquer du code (c'est-à-dire écrire plusieurs fois des morceaux de programme qui font la même chose).

Remarque : il existe déjà des fonctions en Python (built-in) qui font quasiment ce que l'on vous demande : `bin`, `hex` et `int`. Vous n'avez pas le droit de les utiliser dans cet exercice.

Aide : on vous fournit le code pour convertir un nombre en chiffre hexadécimal et vice et versa.

1. Écrivez une fonction `decimal_vers_binaire(n)` qui renvoie une chaîne de caractères correspondant à la représentation binaire de l'entier `n`.
2. Écrivez une fonction `decimal_vers_binaire_normalise(n,k)` qui renvoie une chaîne de caractères correspondant à la représentation binaire de l'entier `n` sur `k` bits. Attention : si `n` ne peut pas s'écrire sur `k` bits, votre fonction devra provoquer une erreur (utilisez `assert`).
3. Écrivez une fonction `binaire_vers_decimal(b)` qui renvoie l'entier correspondant à la représentation binaire sous forme de chaîne de caractères `b`. Attention : si `n` n'est pas écrit en binaire, votre fonction devra provoquer une erreur.
4. Écrivez une fonction `tous_binaire(k)` qui affiche, dans l'ordre, toutes les représentations binaires (normalisées) sur `k` bits.
5. Écrivez une fonction `decimal_vers_hexadecimal(n)` qui renvoie une chaîne de caractères correspondant à la représentation hexadécimale de l'entier `n`.

Remarque : dans le fichier, on vous fournit une fonction `nombre_vers_lettre` qui permet de convertir un entier entre 0 et 15 en un caractère hexadécimal.

6. Écrivez une fonction `hexadecimal_vers_binaire(h)` qui renvoie la représentation binaire correspondant à la représentation hexadécimale `h`.

Remarque : on vous fournit également une fonction `lettre_vers_nombre` qui permet de convertir un caractère hexadécimal en entier entre 0 et 15.

Exercice 3

Pour cet exercice, vous devrez compléter le fichier `exo3.py` fourni.

On souhaite écrire les quatre fonctions listées ci-dessous. Ce sont toutes des fonctions concernant des motifs dans des chaînes de caractères (un motif est une portion de la chaîne : par exemple `pa` et `and` sont des motifs de `panda`, mais pas `pada`). Ces fonctions ont donc des points communs. Pour les écrire, vous devez **commencer par vous demander quelle(s) fonction(s) intermédiaire(s) écrire afin de ne pas refaire quatre fois la même chose**.

1. Écrivez cette ou ces fonctions, ainsi que les doctests pour les tester.
2. Puis écrivez les quatre fonctions suivantes (remarque : vous n'avez pas le droit d'utiliser les méthodes `find` et `replace` qui existent en Python) :
 - (a) `est_motif(chaine, motif)` qui teste si `motif` est un motif de `chaine` ;
 - (b) `nombre_motif(chaine, motif)` qui renvoie le nombre d'occurrences de `motif` dans `chaine` ;
 - (c) `remplace_caractere(chaine, car, motif)` qui renvoie une chaîne où l'on a remplacé toutes les occurrences du caractère `car` par la chaîne `motif` dans `chaine` ;
 - (d) `remplace_motif(chaine, motif1, motif2)` qui renvoie une chaîne où l'on a remplacé toutes les occurrences de la chaîne `motif1` par la chaîne `motif2` dans `chaine` ;